aie26

Topic 06 : Data Modelling

Part 02 : Column Encoding Scaling

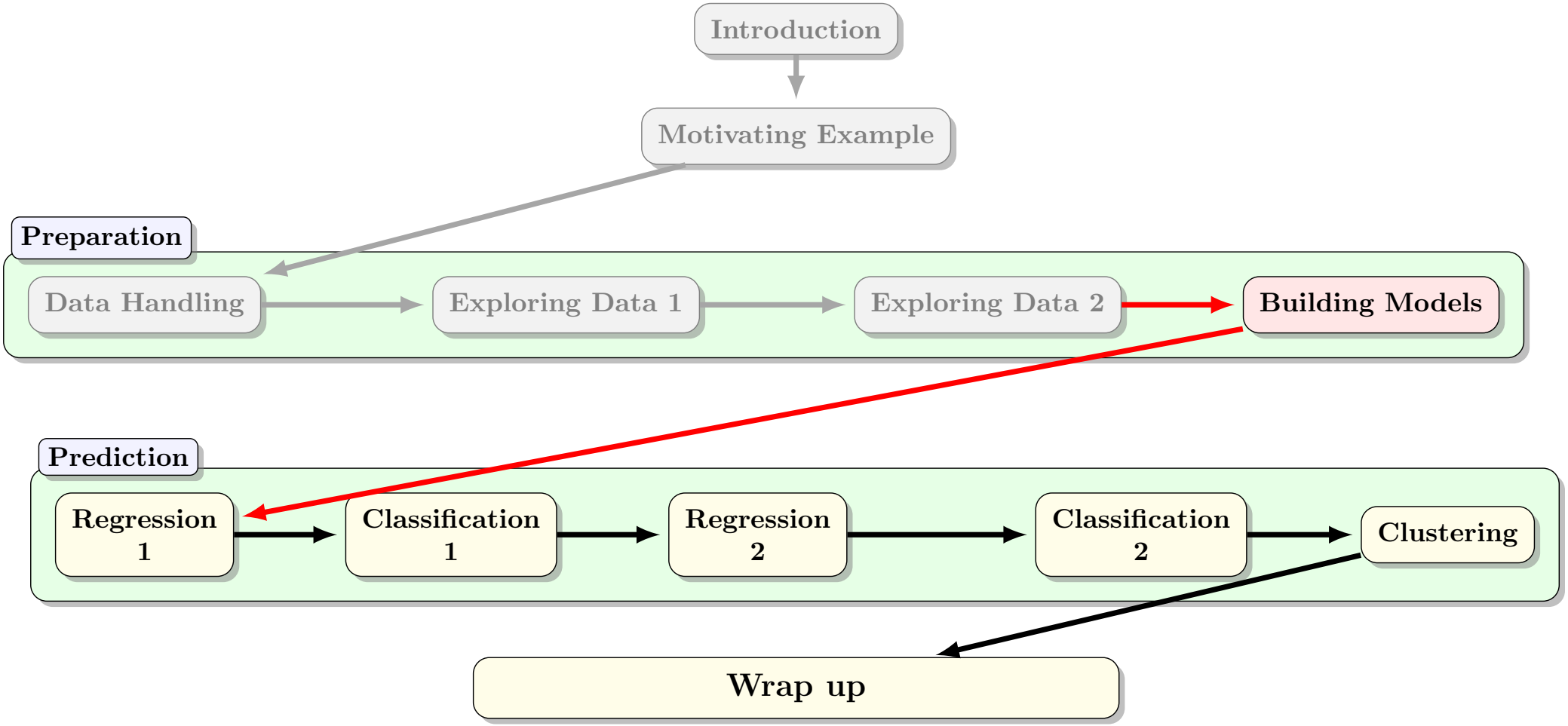
Dr Bernard Butler
Department of Computing and Mathematics, SETU.
(bernard.butler@setu.ie)

Autumn Semester, 2026

Outline

- Encoding categorical features, using pandas
- Scaling numeric features
- Looking ahead to feature preparation
- Overview of ML and what we have achieved

AI Engineering (Week 6)



Outline

1. Modelling Process 3
1.1. Models and error 5

2. Categorical Columns 7

3. Dataset Splits 15

4. Feature engineering 23

5. Wrap up 26

6. Resources 31

Featuring engineering 1: Scaling of numerical variables

Scaling - what it does

- If numeric features have different scales, e.g. [-0.005, -0.003] and [10000, 10001] some terms dominate, others are "lost"
- Better: transfer the scaling from the feature to the model parameter
- A min-max scaling is often a good choice:

$$\tilde{X} = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

- Note that X is in the range $[X_{\min}, X_{\max}]$ but $\tilde{X}$ is in the range $[0, 1]$.
- Other options include StandardScaler (subtract mean and divide by standard deviation) and a max-abs scaler (scales to [-1,1])
- Whatever scaler is used, it should be trained using the training set only. It can be applied to either training or test set.

sklearn example

```
from sklearn.preprocessing import MinMaxScaler
# df is a dataframe with numeric features
scaler = MinMaxScaler()
dfScaled = scaler.fit(df))
```

`dfScaled` can be used instead of `df` with the advantage that the fitted parameters are more accurate.

Feature Engineering 2: Choice of Features

- How many to include? Use metrics to decide. Will see some when considering regression and classification.
- How do we handle different feature types? Need to encode categorical variables.
- Can we derive new numeric features? Yes, $f' = \log(f)$ etc. is possible

Using Categorical Features in (Logistic) Regression

How can Categorical-valued features participate in linear models?

Given the following fragment of a dataset, where the goal is to predict the salary of employees in a large organisation:

```
df = pd.read_csv('data/team.csv',\
       index_col="Name")
df
```

Name	Role	Skilled	Salary
Alice	Designer	Yes	40000
Bob	Programmer	No	25000
Carol	Tester	No	30000

How can this data be represented by a linear model, where all quantities must take numeric values?

Using pandas .getdummies() on a binary-valued column

```
dfSkilledDummies = pd.get_dummies(df['Skilled'],\
       prefix='Skilled',\
       dtype=int)
dfSkilledDummies
```

Name	Skilled_No	Skilled_Yes
Alice	0	1
Bob	1	0
Carol	1	0

Note that a binary-valued column becomes 2 dummy columns

Reducing redundancy (by 1) in 2 dummy columns

```
dfSkilledIndicators = pd.get_dummies(df['Skilled'],\
       prefix='Skilled',\
       drop_first=True,\
       dtype=int)\
       .rename(columns={"Skilled_Yes": "IsSkilled"})
dfSkilledIndicators
```

	IsSkilled
Name	
Alice	1
Bob	0
Carol	0

A single indicator column can replace a group of 2 dummy columns

Using pandas .getdummies() on a multi-valued column

```
dfRoleDummies = pd.get_dummies(df['Role'],\
        prefix='Role',\
        dtype=int)
dfRoleDummies
```

Name	Role_Designer	Role_Programmer	Role_Tester
Alice	1	0	0
Bob	0	1	0
Carol	0	0	1

Note that an n-valued column becomes n dummy columns

Reducing redundancy (by 1) in n dummy columns

```
dfRoleIndicators = pd.get_dummies(df['Role'],\
       prefix='Role',\
       drop_first=True,\
       dtype=int)\
       .rename(columns={\
       "Role_Programmer": "IsProgrammer",\
       "Role_Tester": "IsTester"})
dfRoleIndicators
```

Name	IsProgrammer	IsTester
Alice	0	0
Bob	1	0
Carol	0	1

$n - 1$ indicator columns can replace a group of n dummy columns

Deriving and using dummy/indicator features

- Identify potential categorical features in EDA Pass 1
- Identify whether each feature is (potentially) *usable* in EDA Pass 2
- Identify whether each feature is (potentially) *useful* in EDA Pass 3
- Add all potentially usable and useful features (regardless of type) to a list F
- For each categorical feature f_j in F having n levels
  - Derive $n-1$ indicator features $\tilde{f}_j^k$, where $k = 1, \ldots, n-1$
  - Replace the original categorical feature f_j in F with the derived indicator features $\tilde{f}_j^k$.
- Build the model using the features in F.

Analysis of pandas dummies

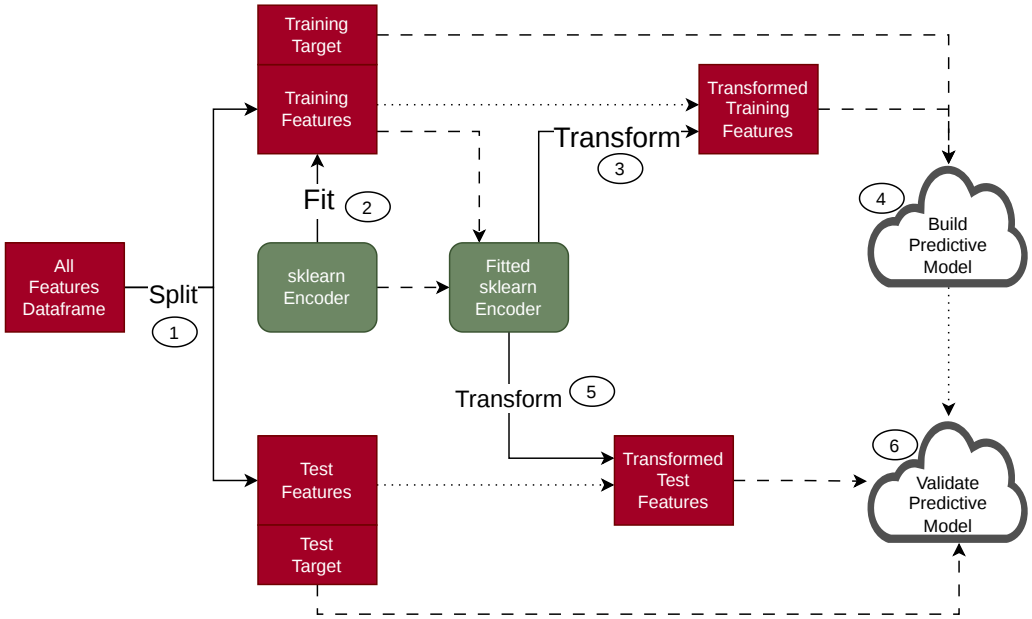
Pandas enables us to convert unordered categorical features into sets of (0,1)-valued numeric features

But what about. . .

1. Ordered categorical features - can we do better than treating them as unordered (and losing information)?
2. Categorical targets (whether ordered or not) - how should we handle these?
3. How do we handle data (that includes categorical columns) that is split into training and test?
4. How can we reverse the operation (i.e., return from (0,1)-valued columns to categorical columns)?

Flow of operations with scikit-learn encoders

- The numbers indicate the flow of operations when scikit-learn's built-in Encoders are used with features.
- As with scaling, the Encoder is "trained" using `.fit()` on the training features only.
- The fitted encoder is applied to the training features using `.transform()` before they are used to build the model.
- The same procedure is applied to the test features so they can be used with the model.
- The transformed features *replace* the original features directly.
- scikit-learn Encoders also provide `.inverse_transform()` to restore the original features.



Worked example: Viewing the dataset

The dataset has a mixture of columns that need to be encoded

- Unordered feature category: `colour`, taking values `blue`, `green` and `red`.
- Ordered feature category: `size`, taking values `small`, `medium` and `large`.
- Date/time feature category: `hour`, taking values from 0 to 23.
- Labeled target: `move`, taking values `down`, `left`, `right` and `up`.

	hour	colour	size	move
32	8	green	medium	left
18	18	blue	small	left
17	17	blue	large	up
33	9	red	large	up
27	3	blue	large	right

Worked example: using `OneHotEncoder`

```
def applyOHEdirect(oheEnc, df, colName,\
                   categories):
  # Apply mapping
  ohe = oheEnc.transform(df[[colName]])
  colNames = ['is_' + x for x in categories]
  dfOHEsk = pd.DataFrame(ohe.toarray(),\
        columns=colNames,\
        index=df.index)
  return dfOHEsk
```

```
colName = 'colour'
# dummies are ints
oheEnc = skp.OneHotEncoder(dtype=int)
# "learn" mapping to dummies
oheEnc.fit(dfTrain[[colName]])
```

```
categories = oheEnc.categories_
dfOheColoursSk = applyOHEdirect(oheEnc,\
        dfTrain, colName, categories)
dfOheColoursSk.head()
```

- `fit` the `OneHotEncoder` to the training data, to learn the mapping from the training set
- `transform` the data (training data is used here, but it could also be applied to test data)
- Note that we try to make the transformed columns easier to interpret

Worked example: `OneHotEncoder` results and inverse

	is_blue	is_green	is_red
32	0	1	0
18	1	0	0
17	1	0	0
33	0	0	1
27	1	0	0

```
# Complete the round trip from unordered category
# to OHE and back
colourOheArr = dfOheColoursSk.values
dfColourRoundTrip = pd.DataFrame(\
    oheEnc.inverse_transform(dfOheColoursSk),\
    index=dfTrain.index,\
    columns=[colName])
dfColourRoundTrip.head()
```

	colour
32	green
18	blue
17	blue
33	red
27	blue

- The transformed OHE colour columns have 0,1 values instead of colour names
- The `inverse_transform` restores the original (`blue`, `green`, `red`) labels (compare with the original training data on earlier slide)

Worked example: using `OrdinalEncoder`

```
# Derive integers in the right order for
# the ordinal category
colName = 'size'
categories=[['small','medium','large']]
ordinalEnc = skp.OrdinalEncoder(\
    categories=categories, dtype=int)
sizeArr = dfTrain[colName]\
    .values.reshape(-1,1)
ordinalEnc.fit(sizeArr)
```

```
ordName = 'ord_' + colName
dfSizeOrdinal = pd.DataFrame(\
    ordinalEnc.transform(sizeArr),\
    index=dfTrain.index,\
    columns=[ordName])
dfSizeOrdinal.head()
```

- `fit` the `OrdinalEncoder` to `size` in the training data, to learn the mapping from the training set
- As before, `transform` the data (training data is used here, but it could also be applied to test data)
- We distinguish between the transformed columns and the original `size` column

Worked example: `OrdinalEncoder` results and inverse

	ord_size
32	1
18	0
17	2
33	2
27	2

```
# Complete the round trip from ordinal to
# integer and back
sizeOrdinalArr = dfSizeOrdinal.values
dfSizeOrdinalRoundTrip = pd.DataFrame(\
    ordinalEnc.inverse_transform(\
        sizeOrdinalArr),\
    index=dfTrain.index,\
    columns=[colName])
dfSizeOrdinalRoundTrip.head()
```

	size
32	medium
18	small
17	large
33	large
27	large

- The transformed Ordinal `size` columns have 0,1,2 values instead of size labels
- The `inverse_transform` restores the original (`small`, `medium`, `large`) labels (compare with the original training data on earlier slide)

Worked example: using `LabelEncoder`

```
# Apply label encoder to the target.
# NB: this encoder is _intended for target
# columns only_.
# DO NOT USE label encoder on features!!!!
colName = 'move'
labelEnc = skp.LabelEncoder()
y = dfTrain[colName]
# See https://stackoverflow.com/a/36120015/1988855
labelEnc.fit(y)
```

```
lblName = 'lbl_' + colName
dfMoveLbl = pd.DataFrame(\
    labelEnc.transform(y),\
    index=dfTrain.index,\
    columns=[lblName])
dfMoveLbl.head()
```

- The `LabelEncoder` should be used with target columns only.
- Never use `LabelEncoder` with feature columns!!!

Worked example: `LabelEncoder` results and inverse

	lbl_move
32	1
18	1
17	3
33	3
27	2

```
# Complete the round trip from label
# to integer and back
yE = dfMoveLbl['lbl_move']
dfMoveLblRoundTrip = pd.DataFrame(\
    labelEnc.inverse_transform(yE),\
    index=dfTrain.index,\
    columns=[colName])
dfMoveLblRoundTrip.head()
```

	move
32	left
18	left
17	up
33	up
27	right

- The Label-encoded transformed target (`move`) columns have 0,1,2,3 values instead of `move` labels
- The `inverse_transform` restores the original (`down`, `left`, `right`, `up`) labels (compare with the original training data on earlier slide)

Bonus 1: manual `CyclicEncoder`

```
def cyclicEncode(inDf, colName, period):
  encoded = {
  colName+'_cos' : np.cos(2 * np.pi * inDf[colName]/period),
  colName+'_sin' : np.sin(2 * np.pi * inDf[colName]/period)
  }
  outDf = pd.DataFrame(encoded, index=inDf.index)
  return outDf
```

```
encodedHour = cyclicEncode(dfTrain, 'hour', 24)
encodedHour.head()
```

	hour_cos	hour_sin
32	-5.000000e-01	0.866025
18	-1.836970e-16	-1.000000
17	-2.588190e-01	-0.965926
33	-7.071068e-01	0.707107
27	7.071068e-01	0.707107

- Cyclic encoding is intended for quantities that "wrap around", like hours, months, etc.
- The formula is well-defined and its parameters do not need to be learnt from the training data
- It can be applied directly as shown, but would need to be adjusted to have the same API as a sklearn equivalent encoder.

Bonus 2: manual `ReverseCyclicEncoder`

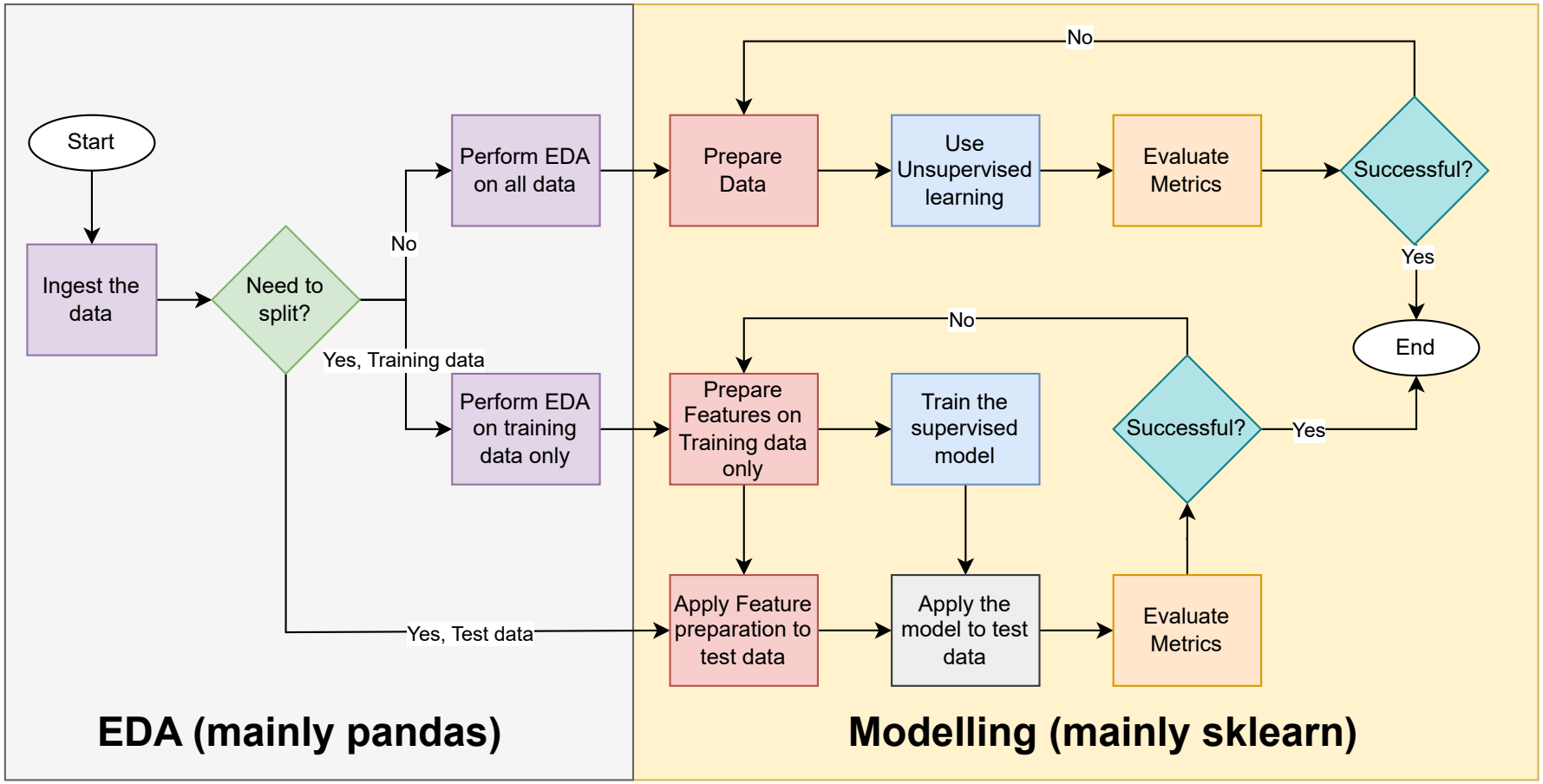
```
def reverseCyclicEncode(inDf, colName, period):
  c = inDf[colName+'_cos'].to_numpy()
  s = inDf[colName+'_sin'].to_numpy()
  theta = np.atan2(s,c); twopi = 2.0*np.pi
  # Ensure theta angle is between 0 and 2*pi
  thetaNN = np.where(theta < 0, theta+twopi, theta)
  decoded = {
    colName+'_inv' : (thetaNN * period / twopi)
  }
  outDf = pd.DataFrame(decoded, index=inDf.index)
  return outDf
```

```
decodedHour = reverseCyclicEncode(encodedHour, 'hour', 24)
decodedHour.head()
```

	hour_inv
32	8.0
18	18.0
17	17.0
33	9.0
27	3.0

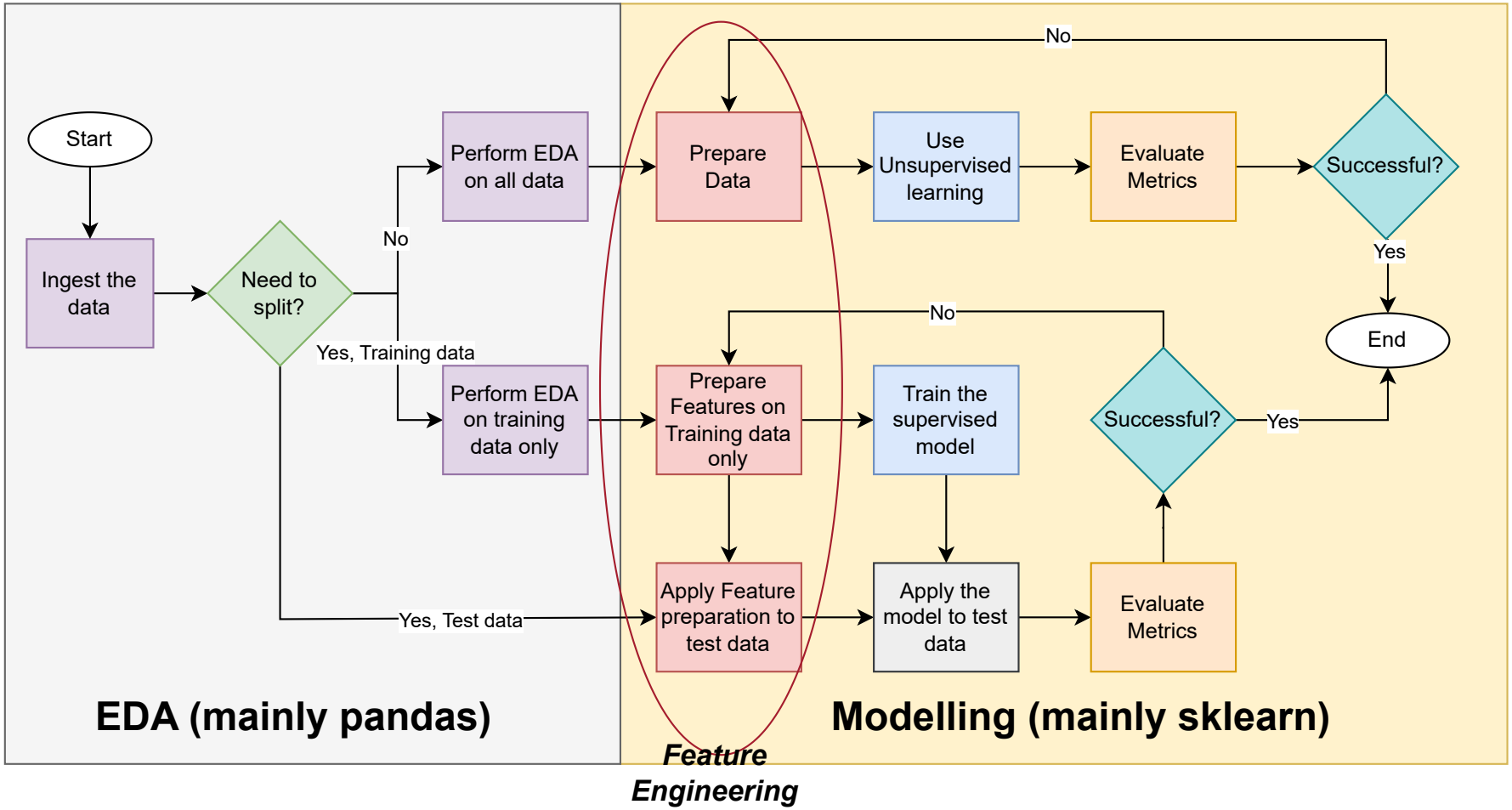
- Reverse cyclic encoding inverts the cyclic encoding, back to the original variable (`hour`) numbers.
- It also does not need to be "learnt" from data, just applied as an expression.
- Unlike OHE, Ordinal and Label encoding, the cyclic encoding round trip does not return exactly the same values, due to roundoff error when computing trigonometric functions.

An overview of Machine Learning



On first glance, this might seem overwhelming, but note that the boxes are colour-coded, so that related operations share the same colour.

Where feature preparation fits. . .



NB: Feature preparation is informed by EDA *but is not part of EDA*. So, not part of your EDA assignment!

Feature Engineering more generally

- Use of pandas `getdummies()` requires care and is not always suitable
- Scikit-learn, imported as the `sklearn` package, supports a variety of column transformations
- Categorical columns can be features or targets, ordered or unordered
- Can also rescale numerical columns, or encode more exotic columns (other datatypes, computed columns, . . .)
- As seen in the schematic, if an ML procedure is unsuccessful, more feature engineering should be considered - it can help.

Summary

- We have reviewed different types of models and considered their general form.
- We looked at the goals of modelling: minimise predictive error.
- We considered how feature engineering can help.
  - Scaling numerical features, so that variation is treated fairly between features.
  - Choosing a subset of features (more to come in future weeks...), looking for the sweet spot between under- and over-fitting.
  - Encoding categorical features as numerical dummy features (more to come in future weeks...), so they can participate in linear models
- In subsequent weeks we will put this theory into practice.

Resources

- **A Summary of the Basic Machine Learning Models**
  `towardsdatascience.com/a-summary-of-the-basic-machine-learning-models-e0a65627ecbe`

- **Train-Test Split for Evaluating Machine Learning Algorithms**
  `https://machinelearningmastery.com/train-test-split-for-evaluating-machine-learning-algorithms`
  This week I have focused on the theory rather than its (python) implementation. This is a nice article that covers the implementation side of things.

- **Cross-Validation: Estimator Evaluator**
  `medium.com/swlh/cross-validation-estimator-evaluator-897d28afb4ff`
  Nice article that covers cross-validation in a lot more detail — we will be using many of these variants in later weeks, especially k-fold stratified.